



Análisis de Script Bash

Script completo

```
#!/bin/bash

echo -e "Paciente\tLongitud\tPos103\tPos239" > ../resultados/informe_final.tsv

for prot in $(grep "^>" ../datos/pacientes.fasta | sed 's/>//'); do
    # Extraer la secuencia completa
    secuencia=$(grep -A1 "^>$prot" ../datos/pacientes.fasta | tail -n1)
    # Calcular longitud
    longitud=${#secuencia}
    # Extraer posición 103
    aa103=$(echo "$secuencia" | cut -c103)
    # Extraer posición 239
    aa239=$(echo "$secuencia" | cut -c239)
    # Guardar en el informe
    echo -e "${prot}\t${longitud}\t${aa103}\t${aa239}" >> ../resultados/informe_final.tsv
done
```

Flujo visual del script

Diagrama de flujo principal

[Inicio]



[Crear archivo de salida con cabecera]



[Buscar todas las cabeceras en FASTA con `grep "^>"`]



[Eliminar el símbolo `>` de cada cabecera con `sed 's/>///'`]



[Iniciar bucle para cada paciente]



Para CADA paciente:

① Buscar cabecera + secuencia con `grep -A1 ">$prot"`

② Extraer solo la secuencia con `tail -n1`

③ Calcular longitud con `${#secuencia}`

④ Extraer carácter en posición `103` con `cut -c103`

⑤ Extraer carácter en posición `239` con `cut -c239`

⑥ Añadir datos al archivo de salida con `echo >>`



[¿Más pacientes?]



└─ Sí ─→ [Volver al inicio del bucle]



└─ No ─→ [Fin]

Esquema detallado del procesamiento de archivos

❖ pacientes.fasta

```
|
|>Paciente_001
|  ATGCGTACGTAGCTAGCTAGCTAGCTAGC...
|
|>Paciente_002
|  GCTAGCTAGCTAGCATCGATCGATCGAT...
|
|>Paciente_003
|  TCGATCGATCGACTGACTGACTGAC...
```



PASO 1: `grep ">"` → Obtiene líneas que empiezan con >

```
>Paciente_001
>Paciente_002
>Paciente_003
```



PASO 2: `sed 's/>/'` → Elimina el carácter >

```
Paciente_001
Paciente_002
Paciente_003
```



BUCLE FOR: Itera sobre cada nombre

```
prot = Paciente_001
prot = Paciente_002
prot = Paciente_003
```



Para CADA paciente:

① `grep -A1 ">${prot}"` → busca cabecera + 1 línea después

```
>Paciente_001
ATGCGTACGTAGCTAGCTAGCTAGCTAGC...
```

- ② `tail -n1` → toma solo la última línea (la secuencia)

```
ATGCGTACGTAGCTAGCTAGCTAGCTAGC...
```

- ③ `${#secuencia}` → calcula la longitud (número de caracteres)

```
150
```

- ④ `cut -c103` → extrae el carácter en posición 103

```
A
```

- ⑤ `cut -c239` → extrae el carácter en posición 239

```
G
```

- ⑥ `echo` → guarda en archivo TSV

```
Paciente_001    150    A    G
```



- ◇ `informe_final.tsv`

Paciente	Longitud	Pos103	Pos239
Paciente_001	150	A	G
Paciente_002	238	C	T
Paciente_003	245	G	A

Análisis línea por línea

Línea 1: Shebang

```
#!/bin/bash
```

Componente	Explicación
#!	Shebang - indica qué intérprete usar
/bin/bash	Ruta al intérprete Bash
Qué hace	Asegura que el script se ejecute con Bash, no con otro shell

Línea 3: Crear archivo de salida

```
echo -e "Paciente\tLongitud\tPos103\tPos239" > ../resultados/informe_final.tsv
```

Componente	Explicación
echo	Comando para imprimir texto
-e	Habilita interpretación de caracteres especiales
\t	Tabulador (caracter especial para separar columnas)
>	Redirección: sobrescribe el archivo de salida
../resultados/	Directorio superior (padre), carpeta resultados
.tsv	Extensión para Tab-Separated Values

Qué hace: Crea un archivo con una fila de cabecera para el informe.

Línea 5: Inicio del bucle

```
for prot in $(grep "^>" ../datos/pacientes.fasta | sed 's/>//'); do
```

Componente	Explicación
for prot in	Inicia un bucle que itera sobre una lista
\$(...)	Sustitución de comando - ejecuta y reemplaza por su salida
grep "^>"	Busca líneas que empiezan con >
^>	Expresión regular: ^ = inicio de línea, > = carácter literal
../datos/ pacientes.fasta	Ruta al archivo FASTA de entrada
	Pipe - conecta salida de un comando a la entrada del siguiente
sed 's/>/'	Sustituye > por nada (lo elimina)
's/>/'	Sustitución: s/patrón/reemplazo/
do	Marca el inicio del bloque del bucle

Qué hace: Extrae todos los nombres de pacientes del archivo FASTA y los usa para iterar en el bucle.

Línea 7: Extraer secuencia

```
secuencia=$(grep -A1 "^>$prot" ../datos/pacientes.fasta | tail -n1)
```

Componente	Explicación
secuencia=	Asigna el resultado a la variable secuencia
grep -A1	-A1 = muestra 1 línea después de la coincidencia
"^>\$prot"	Busca la línea que empieza con > seguido del nombre del paciente
tail -n1	-n1 = muestra solo la última línea
\$prot	Variable que contiene el nombre del paciente actual

Qué hace: Busca la cabecera del paciente en el FASTA, toma la línea siguiente (la secuencia) y la guarda.

Ejemplo de ejecución:

- Entrada: `grep -A1 "^>Paciente_001" pacientes.fasta`
- Salida:

```
Paciente_001
ATGCGTACGTAGCTAGC...
```

- `tail -n1` se queda con: `ATGCGTACGTAGCTAGC...`

Línea 9: Calcular longitud

```
longitud=${#secuencia}
```

Componente	Explicación
<code>\${#variable}</code>	Sintaxis de Bash para obtener la longitud de una variable
secuencia	Variable que contiene la secuencia de ADN

Qué hace: Calcula el número de caracteres de la secuencia.

Ejemplo:

- secuencia="ATCG" (4 caracteres)
- longitud=\${#secuencia} (longitud = 4)

Línea 11: Extraer posición 103

```
aa103=$(echo "$secuencia" | cut -c103)
```

Componente	Explicación
echo "\$secuencia"	Imprime el contenido de la variable
	Pipe: pasa la salida al siguiente comando
cut -c103	-c = selecciona caracteres, 103 = posición 103

Qué hace: Extrae el carácter que está en la posición 103 de la secuencia.

Ejemplo:

- echo "ABCDEFGHJKLMNOPQRSTUVWXYZ" | cut -c5
- Devuelve: E

Línea 13: Extraer posición 239

```
aa239=$(echo "$secuencia" | cut -c239)
```

Igual que la línea 11 pero para la posición 239.

Línea 15: Guardar en informe

```
echo -e "${prot}\t${longitud}\t${aa103}\t${aa239}" >> ../resultados/informe_final.tsv
```

Componente	Explicación
echo -e	Imprime con interpretación de caracteres especiales
\${prot}	Variable con el nombre del paciente
\t	Tabulador entre columnas
>>	Redirección append: añade al final del archivo
../resultados/ informe_final.tsv	Archivo de salida

Qué hace: Añade una nueva fila al informe con los datos del paciente actual.

Línea 16: Fin del bucle

```
done
```

Marca el final del bloque for.

Manual de referencia de comandos

grep - Global Regular Expression Print

Sintaxis básica:

```
grep [opciones] patrón [archivo]
```

Opciones usadas en el script:

Opción	Significado	Ejemplo
^	Inicio de línea (expresión regular)	grep "^>" busca líneas que empiezan con >
-A n	Muestra n líneas después de la coincidencia	grep -A1 ">" muestra la coincidencia y 1 línea después

Otras opciones útiles:

Opción	Uso
-i	Ignora mayúsculas/minúsculas
-v	Muestra líneas que NO coinciden
-c	Cuenta coincidencias (no muestra líneas)
-B n	Muestra n líneas antes de la coincidencia
-C n	Muestra n líneas antes y después

Manual: `man grep`

sed - Stream Editor

Sintaxis básica:

```
sed 'comando' [archivo]
```

Comando usado en el script:

Sintaxis	Significado
<code>s/>/</code>	Sustituir: reemplaza > por nada
<code>s/patrón/reemplazo/</code>	Formato general de sustitución

Ejemplos adicionales:

Comando	Efecto
sed 's/A/T/g'	Reemplaza todas las A por T (g = global)
sed 's/^>/'	Elimina el > al inicio de línea
sed '3d'	Elimina la línea 3
sed 's/^/>> /'	Añade ">> " al inicio de cada línea

Manual: `man sed`

cut - Extraer secciones de líneas

Sintaxis básica:

```
cut [opciones] [archivo]
```

Opciones usadas en el script:

Opción	Significado	Ejemplo
-c n	Selecciona el carácter en posición n	cut -c103

Otras opciones útiles:

Opción	Uso
-f n	Selecciona el campo n (con -d para delimitador)
-d','	Especifica el delimitador (por defecto tabulador)
-c n-m	Selecciona caracteres desde n hasta m
-c n-	Selecciona desde la posición n hasta el final

Ejemplos prácticos:

- `echo "uno,dos,tres" | cut -d',' -f2` → Devuelve: dos
- `echo "ABCDE" | cut -c2-4` → Devuelve: BCD
- `echo "ABCDE" | cut -c3-` → Devuelve: CDE

Manual: `man cut`

Expansión de variables en Bash

Sintaxis	Significado	Ejemplo
<code>\${#var}</code>	Longitud de la variable	<code>\${#secuencia}</code>
<code>\${var:pos}</code>	Subcadena desde posición	<code>\${secuencia:102:1}</code>
<code>\${var:pos:len}</code>	Subcadena de longitud len	<code>\${secuencia:102:1}</code>

Importante: En Bash, el conteo empieza en 0, por eso posición - 1.

Redirecciones y tuberías

Símbolo	Nombre	Uso
<code>></code>	Redirección	Sobrescribe archivo
<code>>></code>	Redirección append	Añade al final del archivo
<code><</code>	Redirección entrada	Lee desde archivo
<code> </code>	Pipe	Conecta salida de un comando a entrada de otro
<code>2></code>	Redirección errores	Redirige stderr

Ejemplos:

- comando > archivo.txt → Guarda salida estándar
- comando >> archivo.txt → Añade al archivo
- comando < archivo.txt → Lee desde archivo
- comando1 | comando2 → Pasa salida de comando1 a comando2
- comando 2> errores.txt → Guarda errores

Resumen rápido de comandos

Comando	Uso básico	En el script
grep	Buscar texto en archivos	Buscar cabeceras FASTA (^>)
sed	Editar texto en flujo	Eliminar > de cabeceras
cut	Extraer partes de texto	Extraer caracteres por posición
echo	Imprimir texto	Crear líneas del informe
	Tubería	Conectar comandos
>	Redirección	Guardar salida en archivo
>>	Redirección append	Añadir al archivo
\${#var}	Longitud de variable	Calcular tamaño de secuencia
\$(comando)	Sustitución de comando	Ejecutar y usar resultado

Referencias

- Manual de grep: man grep o GNU Grep (<https://www.gnu.org/software/grep/manual/>)
- Manual de sed: man sed o GNU Sed (<https://www.gnu.org/software/sed/manual/>)

- Manual de cut: man cut o GNU Coreutils (https://www.gnu.org/software/coreutils/manual/html_node/cut-invocation.html)
- Bash Guide: Bash Reference Manual (<https://www.gnu.org/software/bash/manual/>)
- FASTA format: NCBI FASTA (<https://www.ncbi.nlm.nih.gov/genbank/fastaformat/>)